

Könyvtári Kölcsönző Rendszer

Fejlesztői Dokumentáció

Verzió: 2.2

Dátum: 2026. január 11.

Státusz: Véglegesítve

Fejlesztők:

Káldor Levente Viktor

Lőrincz Márton

Csikós Bendegúz

Juhász Arnold

Projektvezető, Szoftverfejlesztő, Rendszertervező

UI Tervező, Kivételkezelő mester

UX Tervező, Segítő, Váll Feletti Tesztelő mester

Ütemtervező, SCRUM Master

1. Bevezetés

A **Könyvtári Kölcsönző Rendszer** (Library Loan System) egy Java alapú, parancssoros (CLI) alkalmazás, amelynek célja egy kis- és közepes méretű könyvtár alapvető folyamatainak automatizálása. A szoftver kezeli a könyvvállományt, a tagok nyilvántartását, a kölcsönzési folyamatokat, valamint a késedelmes visszavételek utáni büntetések kiszámítását. A felhasználói felület ANSI színkódokkal támogatott terminál ablakban fut.

2. Rendszerarchitektúra

Az alkalmazás egy klasszikus többretegű architektúrát követ, szigorúan elválasztva a megjelenítést, az üzleti logikát és az adattárolást.

Rétegek felépítése:

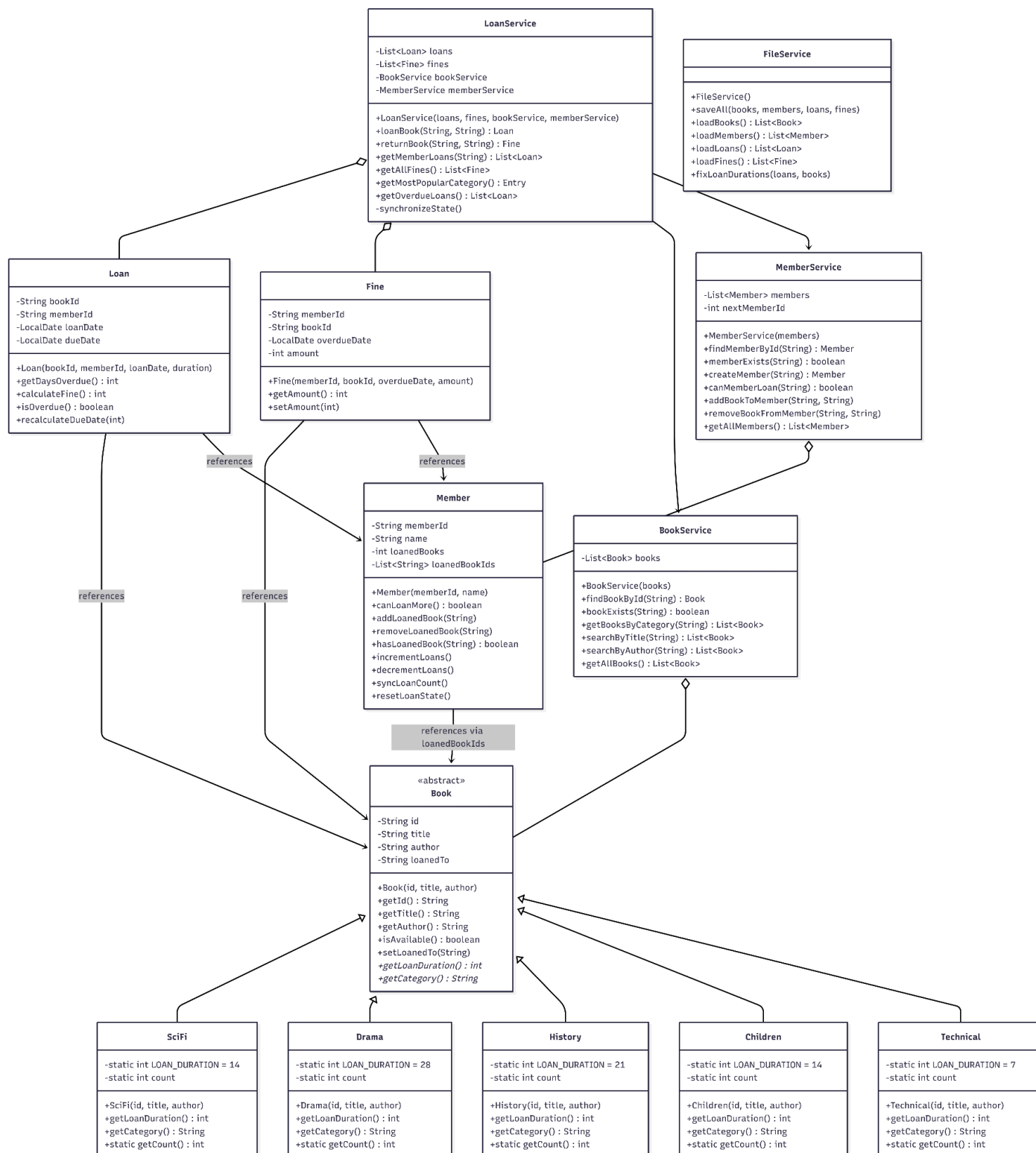
1. **UI Réteg (main.ui):** A felhasználóval való kapcsolattartásért felel. Fogadja a bemeneteket és megjeleníti az adatokat.
2. **Szolgáltatás Réteg (main.service):** Tartalmazza az üzleti logikát (pl. keresés, validáció, büntetés számítás). Ez a réteg köti össze a modellt a perzisztenciával.
3. **Modell Réteg (main.model):** Az üzleti entitásokat (POJO) és azok állapotát leíró osztályok.
4. **Perzisztencia / Adat Réteg:** A FileService osztályon keresztül valósul meg, amely szöveges fájlokban tárolja az állapotot.

Ábrák (charts):

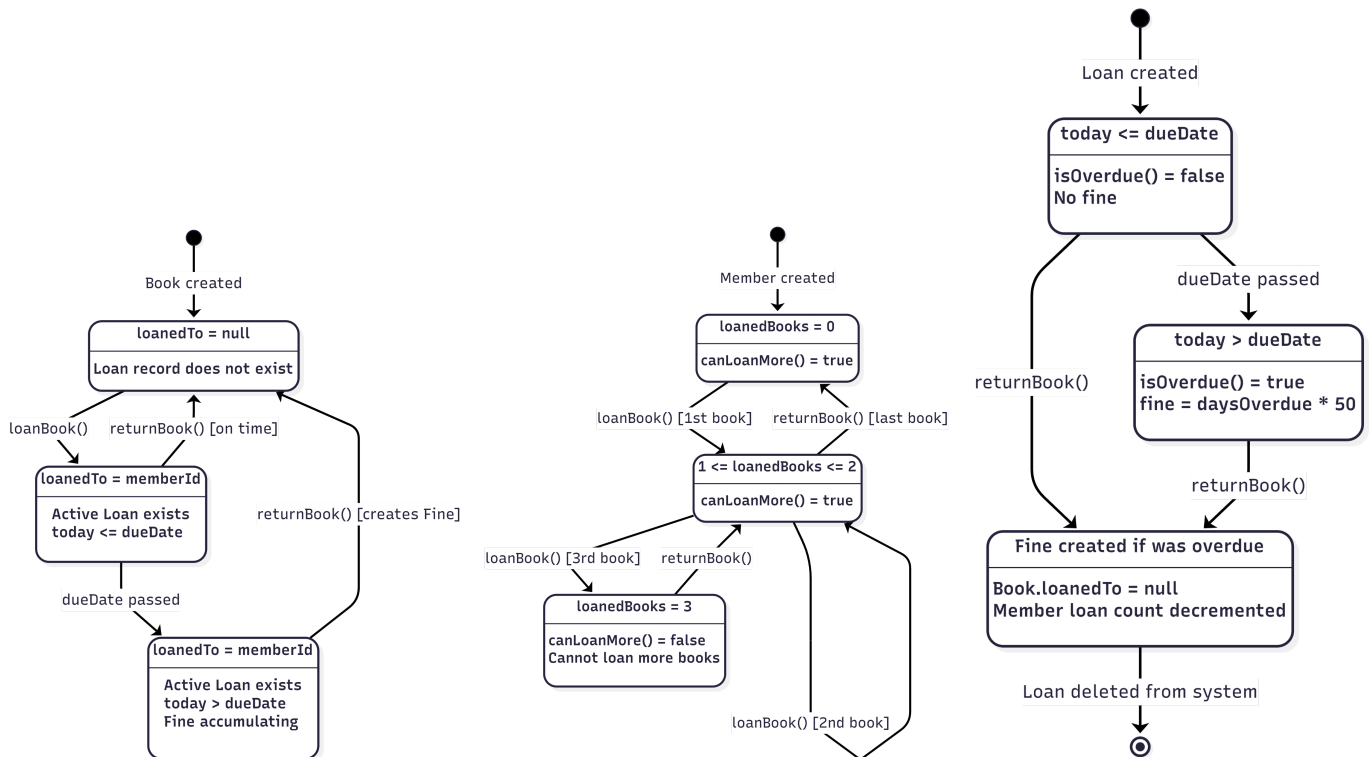
A Rendszer felépítését szemléltető képek.

Folyamatábra: túl nagy a kép, Lásd: [folyamatábra.png](#),

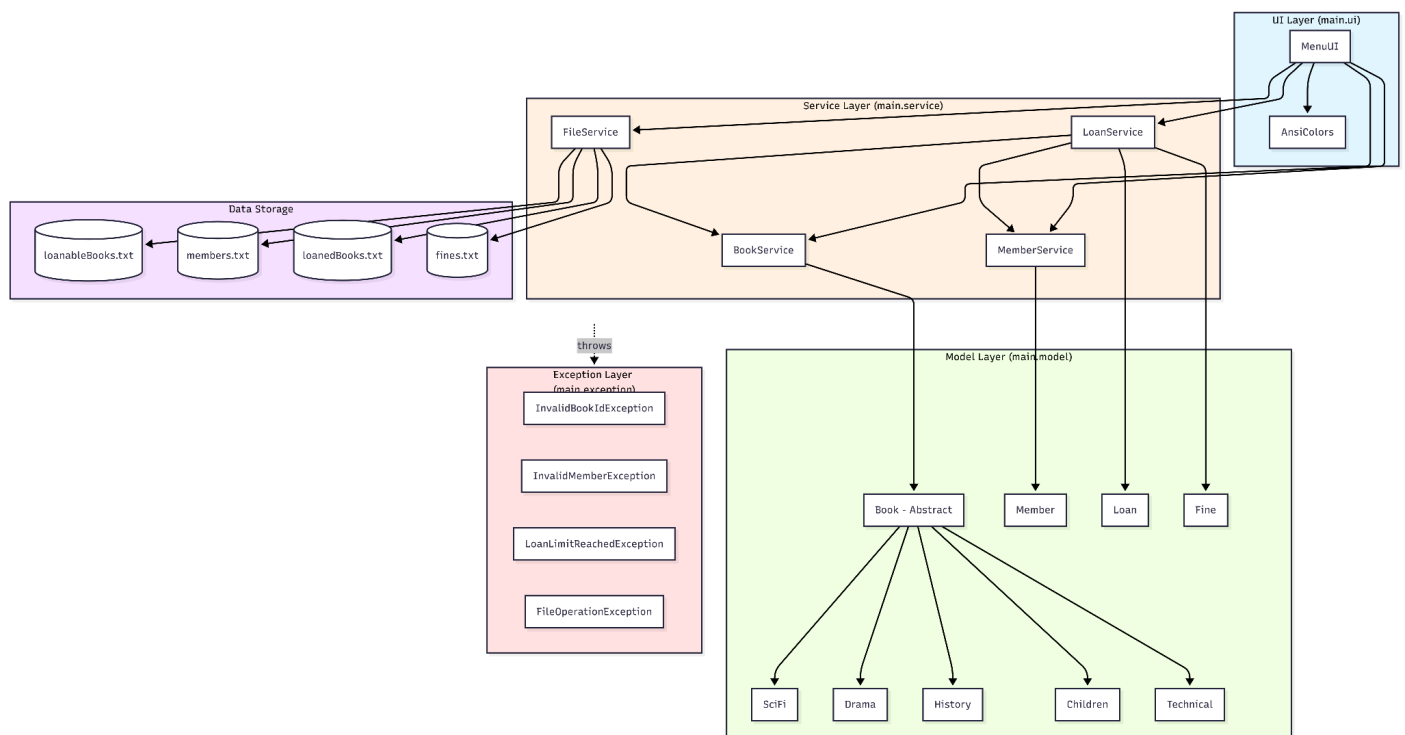
UML Osztálydiagramam:



Állapotdiagramok (3db):



Komponensdiagram:



3. Adatmodell és Osztályszerkezet (main.model)

A modell réteg definiálja a rendszer központi objektumait. Kiemelt szerepet kap az öröklődés a könyvtípusok kezelésénél.

3.1. Könyvek (Book és leszármazottai)

A Book egy absztrakt őszosztály, amely definiálja az alapvető tulajdonságokat (ID, cím, szerző, kölcsönzés állapota). A konkrét kölcsönzési időtartamokat a leszármazott osztályok határozzák meg polimorfizmus segítségével.

Book (Absztrakt):

Szerepe: A könyvtár könyveinek általános reprezentációja

Tulajdonságok:

- `id` (final String) - Egyedi azonosító, kategória prefixszel (pl. "SF-042")
- `title` (final String) - A könyv címe
- `author` (final String) - A szerző neve
- `loanedTo` (String) - Ki kölcsönözte ki (null ha elérhető)

Viselkedés:

- `isAvailable()` - Elérhető-e a könyv (`loanedTo == null`)
- `setLoanedTo(String)` - Kikölcsönzés állapot beállítása
- `getLoanDuration()` - ABSZTRAKT: kategóriánként eltérő
- `getCategory()` - ABSZTRAKT: kategória neve

Öröklődési hierarchia:

Book (absztrakt)

```
├── SciFi (14 nap, "Sci-fi")
├── Drama (28 nap, "Drama")
├── History (21 nap, "History")
├── Children (14 nap, "Children")
└── Technical (7 nap, "Technical")
```

Állapotok:

- ELÉRHETŐ: `loanedTo == null`
- KIKÖLCSÖNÖZVE: `loanedTo != null` (tartalmazza a tag ID-t)

3.2. Tagok (Member)

Szerepe: Könyvtári tag, aki könyveket kölcsönözhet

Tulajdonságok:

- `memberId` (String) - 6 számjegyű egyedi azonosító (pl. "000042")
- `name` (String) - Tag neve
- `loanedBooks` (int) - Aktuálisan kikölcsönzött könyvek száma
- `loanedBookIds` (List<String>) - Kikölcsönzött könyvek ID listája

Korlátozások:

- Maximum 3 könyvet kölcsönözhet egyszerre
- Név validáció: 2-100 karakter, csak betűk, szóköz, kötőjel

Viselkedés:

- `canLoanMore()` - Kölcsönözhet-e még (`loanedBooks < 3`)
- `addLoanedBook(String)` - Könyv hozzáadása a listához
- `removeLoanedBook(String)` - Könyv eltávolítása a listából
- `hasLoanedBook(String)` - Kikölcsönözte-e az adott könyvet
- `incrementLoans()` / `decrementLoans()` - Számláló növelése/csökkentése
- `syncLoanCount()` - Lista mérete → számláló szinkronizálás
- `resetLoanState()` - Minden kölcsönzés törlése

Állapotok:

- ÜRES: `loanedBooks = 0`
- AKTÍV: $1 \leq \text{loanedBooks} \leq 2$
- TELJES: `loanedBooks = 3` (nem kölcsönözhet többet)

Életciklus:

1. Létrehozás → `loanedBooks = 0`, `loanedBookIds` = üres lista
2. Kölcsönzés → `incrementLoans()`, `addLoanedBook()`
3. Visszahozás → `decrementLoans()`, `removeLoanedBook()`
4. Szinkronizálás → `syncLoanCount()` (lista mérete = számláló)

3.3. Kölcsönzés (Loan)

Szerepe: Egy konkrét kölcsönzési tranzakció nyilvántartása

Tulajdonságok:

- `bookId` (final String) - Kikölcsönzött könyv ID-ja
- `memberId` (final String) - Kölcsönző tag ID-ja

- `loanDate` (final LocalDate) - Kölcsönzés dátuma
- `dueDate` (LocalDate) - Visszahozási határidő

Viselkedés:

- `getDaysOverdue()` - Hány napja késésben (0 ha időben van)
- `calculateFine()` - Büntetés összege (késés * 50 HUF)
- `isOverdue()` - Késésben van-e (`getDaysOverdue()` > 0)
- `recalculateDueDate(int)` - Határidő újraszámítása helyes időtartammal

Számítások:

- $\text{dueDate} = \text{loanDate} + \text{loanDuration}$ (kategóriától függ)
- $\text{daysOverdue} = \max(0, \text{today} - \text{dueDate})$
- $\text{fine} = \text{daysOverdue} * 50 \text{ HUF}$

Állapotok:

- AKTÍV - IDŐBEN: $\text{today} \leq \text{dueDate}$
- AKTÍV - KÉSÉSBEN: $\text{today} > \text{dueDate}$
- LEZÁRT: visszahozás után (törlődik a loans listából)

Életciklus:

1. Létrehozás → $\text{loanDate} = \text{ma}$, $\text{dueDate} = \text{ma} + \text{duration}$
2. Várakozás → dueDate közeledik
3. Opcionális késés → $\text{isOverdue}() = \text{true}$, fine számítható
4. Visszahozás → Fine objektum létrehozása (ha késés), Loan törlése

3.4. Büntetés (Fine)

Szerepe: Késői visszahozásért járó büntetés nyilvántartása

Tulajdonságok:

- `memberId` (final String) - Ki kapta a büntetést
- `bookId` (final String) - Melyik könyv miatt
- `overdueDate` (final LocalDate) - Mettől késett ($\text{dueDate} + 1 \text{ nap}$)
- `amount` (int) - Büntetés összege HUF-ban

Viselkedés:

- `setAmount(int)` - Összeg módosítása (pl. elengedés esetén)

Számítás:

- $\text{amount} = (\text{ma} - \text{overdueDate}) * 50 \text{ HUF}$
- $\text{overdueDate} = \text{loan.dueDate} + 1 \text{ nap}$

Állapotok:

- AKTÍV: Még nem fizették ki
- TÖRLŐDIK: `clearFine()` vagy `clearAllMemberFines()` után

Életciklus:

1. Létrehozás → `returnBook()` során, ha `isOverdue() = true`
2. Tárolás → fines listában marad
3. Megjelenítés → `listFines()`, `getMemberFines()`
4. Opcionális törlés → `clearFine()` (tag-könyv pár alapján)

3.5. KAPCSOLATOK ÉS FÜGGŐSÉGEK

Book ↔ Loan

- Egy Book több Loan-ban szerepelhet (időben egymás után)
- Egy Loan pontosan 1 Book-hoz tartozik
- `Book.loanedTo` szinkronban van az aktív `Loan.memberId`-vel

Member ↔ Loan

- Egy Member maximum 3 aktív Loan-nal rendelkezhet
- Egy Loan pontosan 1 Member-hez tartozik
- `Member.loanedBookIds` szinkronban van az aktív `Loan.bookId`-kkel

Loan → Fine

- Egy Loan 0 vagy 1 Fine-t generál (visszahozáskor, ha overdue)
- Egy Fine pontosan 1 Loan-ból származik (de a Loan már töröltött)

Member ↔ Fine

- Egy Member több Fine-nal rendelkezhet
- Egy Fine pontosan 1 Member-hez tartozik

3.6. ÁLLAPOT SZINKRONIZÁCIÓ

`synchronizeState()` - a program indulásakor:

1. Minden `Book.loanedTo` → null
2. Minden `Member.resetLoanState()`
3. Minden Loan alapján:
 - `Book.loanedTo = Loan.memberId`
 - `Member.addLoanedBook(Loan.bookId)`
 - `Member.incrementLoans()`

Ez biztosítja, hogy:

- Book állapot ↔ Loan konzisztens
- Member állapot ↔ Loan konzisztens
- Perzisztencia után helyes állapot áll helyre

3.7. ÜZLETI LOGIKA SZABÁLYOK

1. **Kölcsönzés előtt:** Book.isAvailable() ÉS Member.canLoanMore()
2. **Kölcsönzéskor:** Loan létrehozás → Book.loanedTo + Member.addLoanedBook
3. **Visszahozáskor:** Fine generálás (ha késés) → Loan törlés → állapotok nullázása
4. **Büntetés:** 50 HUF/nap, overdueDate-től számítva
5. **Kategória időtartamok:** Technical=7, Children/SciFi=14, History=21, Drama=28 nap

4. Szolgáltatások és Üzleti Logika (main.service)

4.1. FileService (Adatperzisztencia)

Központi osztály a fájlműveletekhez. Induláskor betölti, leálláskor menti az adatokat.

- **Adatkönyvtár:** data/
- **Kezelt fájlok:** loanableBooks.txt, members.txt, loanedBooks.txt, fines.txt.
- **Formátum:** Szemikolonnal (;) elválasztott szöveges adatbázis.
- **Helyreállítás:** Betöltéskor a fixLoanDurations metódus szinkronizálja a kölcsönzési határidőket a könyvtípusok szabályai alapján.

4.2. LoanService (Kölcsönzési logika)

Ez az osztály végzi a legkomplexebb műveleteket.

- **Szinkronizáció:** A konstruktorban futó synchronizeState() metódus biztosítja, hogy a betöltött könyvek és tagok státusza (kölcsönzött-e, kinél van) konzisztens legyen a Loan objektumokkal.
- **Büntetés:** FINE_PER_DAY konstans (50 HUF) alapú számítás.
- **Statisztika:** Képes lekérdezni a leggyakrabban késő tagokat és a kategóriánkénti kölcsönzési számokat.

4.3. BookService és MemberService

CRUD (Create, Read, Update, Delete) jellegű műveletek és keresés.

- **Keresés:** Java Stream API használata a szűrésre (filter), rendezésre (sorted) és gyűjtésre (collect).
- **ID Generálás:** A MemberService automatikusan növekvő ID-t generál (calculateNextMemberId) a meglévő tagok alapján.

5. Felhasználói Felület (main.ui)

A UI réteg a MenuUI osztályban összpontosul.

- **Működés:** Végtelen while ciklusban várja a felhasználói parancsokat.
- **Színezés:** Az AnsiColors segédosztály statikus metódusai (pl. red(), green(), cyan()) biztosítják a vizuális visszajelzést (hibaüzenetek, sikeres műveletek).
- **Kivételkezelés a UI-ban:** A try-catch blokkok itt kapják el az üzleti logikából (Service réteg) érkező specifikus kivételeket (pl. InvalidBookIdException, LoanLimitReachedException), és fordítják le azokat felhasználóbarát hibaüzenetekre.

6. Kivételkezelés

Bár a kivétel osztályok forráskódja triviális, szerepük kulcsfontosságú a folyamatvezérlésben. A rendszer egyedi, ellenőrzött (checked) kivételeket használ az üzleti szabályok megsértésének jelzésére:

- InvalidBookIdException: Nem létező könyv hivatkozása.
- InvalidMemberException: Nem létező tag, vagy törlési kísérlet aktív kölcsönzéssel rendelkező tagnál.
- LoanLimitReachedException: A tag elérte a maximum 3 könyvet.
- FileOperationException: Kritikus hiba a fájlok írása/olvasása során (a program leállítását okozhatja).

7. Telepítési és Futtatási Útmutató

7.1. Követelmények

- **Java Development Kit (JDK):** 21-es vagy újabb verzió.
- **Operációs Rendszer:** Windows 10/11 (a Main.java és MenuUI Windows környezetre optimalizált karakterkódolást és terminálkezelést feltételezhet, bár a Java platformfüggetlen).

7.2. Fordítás és Futtatás

A projekt gyökérkönyvtárából:

1. Fordítás:

Bash

```
javac -d bin -sourcepath src src/main/Main.java
```

2. Futtatás:

Bash

```
java -cp bin main.Main
```

7.3. Fájlrendszer inicializálása

Az első indításkor a FileService automatikusan létrehozza a data/ mappát. A rendszer működéséhez szükséges, hogy a programnak legyen írási joga ebben a könyvtárban.

8. Fejlesztési Ütemterv és Státusz

A projekt az Ütemterv.xlsx alapján a tervezett mérföldköveket teljesítette.

1. **Specifikáció:** Kidolgozva és bővíve.
2. **Implementáció:**
 - Osztályszerkezet: Kész.
 - Fájlkezelés: Kész.
 - Tesztelés: A Main metódus sikeres inicializálási logeket tartalmaz.
3. **Dokumentáció:** Jelen dokumentummal teljesítve.

A szoftver jelen állapotában **kész** fázisban van, minden funkcionális követelmény (kölsönzés, visszavétel, statisztika, perzisztencia) implementálásra került.

9. Ismert Limitációk

- Nincs concurrent access kezelés
- Nincs audit trail (ki, mikor, mit módosított)
- Nincs undo/redo funkció
- Fájl-alapú perzisztencia skálázhatósági korlátai

10. Jövőbeli Fejlesztések (Roadmap)

- Adatbázis integráció (SQLite/H2)
- Grafikus felhasználói felület (JavaFX)
- Webes interfész (Spring Boot)
- Előjegyzési rendszer
- Email értesítések határidő előtt
- Könyv beszerzési javaslatok a statisztikák alapján

Dokumentum Verzió

- **Verzió:** 2.2
- **Dátum:** 2026-01-11
- **Státusz:** Véglegesítve
- **MI Használat:** Az ábrák készítéséhez a Claude AI-t használtuk.